FORMATION

Analyse de données avec Python

Jour 1, matin

Environnement de travail, bases de numpy et prise en main de pandas

Niveau intermédiaire | 3 jours, 6 demi-journées de 3h30 | Cours et TP sur Jupyter

Le programme des trois jours

	Matin	Après-midi
Jour 1	Environnement de travail, bases de numpy, prise en main de pandas	Sélection, transformation et nettoyage de données
Jour 2	Agrégation, dates et automatisation de plusieurs fichiers	Visualisation avec matplotlib, seaborn et plotly
Jour 3	Organisation de projet, code robuste et CLI	Gestion des dépendances avec uv et projet fil rouge

Chaque demi-journée alterne cours et TP. Bonus si le temps le permet : Polars, DuckDB, Streamlit, empaquetage. Évaluation finale par un QCM.

Le déroulé de la matinée

1	2	3	4	5		6	7	8
Accueil	**venv et pip**	**Jupyter**	**Python**	**numpy**	**Pause**	**pandas**	**Fichiers**	**Exploration**
TP0		TP1 partie 0	TP1 partie 1	TP1 partie 2		TP1 partie 3	TP1 partie 4	TP1 partie 5

9h00

début

10h30

pause

12h30

fin

Chaque section de cours est courte et suivie de la partie du TP1 qui lui correspond.

00

Les postes de la formation

Ubuntu | Python 3.12 | TP0

0 Ubuntu et Python 3.12

Python 3.12 est déjà installé : on ne l'installe pas, on crée des environnements qui s'appuient sur lui.

Dans un terminal, hors environnement virtuel

```
python3 --version
# Python 3.12

pip install pandas
# error: externally-managed-environment
```

La commande s'appelle `python3` : `python` n'existe qu'une fois un environnement virtuel activé
Le Python du système est **protégé** : un `pip install` y est refusé

- C'est voulu : Ubuntu s'en sert pour ses propres outils, et rappelle qu'il faut travailler dans un environnement virtuel

Pratique : le TP0. Créer un environnement virtuel, ouvrir Jupyter, quelques exercices de Python, puis installer et importer numpy.

01

L'environnement de travail

venv | pip | requirements.txt

1 Pourquoi un environnement virtuel

Un dossier qui contient son propre Python et ses propres bibliothèques, isolé du Python de la machine.

1

Isolation

Le projet A reste sur pandas 2.2, le projet B passe à pandas 3.0, sans conflit entre eux.

2

Reproductibilité

La liste des bibliothèques est connue et peut être réinstallée à l'identique sur une autre machine.

3

Propreté

Rien n'est installé dans le Python du système, qui peut servir à d'autres outils.

La règle : un projet, un environnement virtuel.

1 Créer et activer l'environnement

Depuis le dossier du projet

```
python3 -m venv .venv
```

Système	Activation
Ubuntu, macOS	`source .venv/bin/activate`
Windows PowerShell	`.venv\Scripts\Activate.ps1`
Windows cmd	`.venv\Scripts\activate.bat`

`(.venv)` apparaît au début de l'invite, et `python` désigne le Python de l'environnement

- Vérifier le Python utilisé :

`which python` sur Ubuntu et macOS
`where python` sur Windows

Sortir de l'environnement : `deactivate`

- PowerShell bloque le script d'activation ?
  - `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Sur Ubuntu, message `ensurepip is not available` ? Il manque le paquet `python3.12-venv` : `sudo apt install python3.12-venv`.

1 Installer des bibliothèques avec pip

```
python -m pip install --upgrade pip

python -m pip install pandas numpy
python -m pip install "pandas==3.0.6"

python -m pip list
python -m pip show pandas
python -m pip uninstall pandas
```

?

Pourquoi `python -m pip` ?

Pour être certain d'installer dans le Python actif, et pas dans un autre Python présent sur la machine.

Erreur classique : installer une bibliothèque dans un Python, puis exécuter le code avec un autre. Résultat : `ModuleNotFoundError`.

1 Le fichier requirements.txt

Figer l'environnement

```
python -m pip freeze > requirements.txt
```

Le recréer ailleurs

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Ou l'écrire à la main

```
pandas>=3.0
numpy>=2.0
jupyterlab
openpyxl
pyarrow
```

`freeze` est précis mais liste aussi toutes les dépendances indirectes. Écrit à la main, le fichier est lisible mais ne fige pas tout.

Le dossier `.venv` ne se partage pas et ne se versionne pas : on le recrée à partir du `requirements.txt`, et il figure dans le `.gitignore`.

02

Jupyter Notebook et JupyterLab

Lancement | Interface | Ordre d'exécution

2 Lancer Jupyter depuis l'environnement

```
python -m pip install jupyterlab notebook

jupyter lab
jupyter notebook
```

Dans le notebook : quel Python ?

```
import sys
sys.executable
```

Lancer depuis le terminal où le `.venv` est **activé**
Lancer depuis le **dossier du projet** : il devient la racine des chemins relatifs comme `data/ventes.csv`
Le serveur tourne dans le terminal : le laisser ouvert, `Ctrl+C` pour l'arrêter

- JupyterLab : explorateur, onglets, terminal. Notebook : vue centrée sur un document

VS Code ouvre aussi les `.ipynb` : choisir le `.venv` comme noyau

2 Premiers repères dans l'interface

- Deux types de cellules :

code, exécutée par le noyau Python
Markdown, pour le texte et les titres

- Deux modes :

édition avec `Entrée`
commande avec `Échap`

La dernière expression d'une cellule s'affiche sans `print`

Raccourci	Action
`Maj+Entrée`	exécuter et passer à la suivante
`Ctrl+Entrée`	exécuter sans avancer
`A` / `B`	insérer au-dessus / en dessous
`M` / `Y`	passer en Markdown / en code
`D D`	supprimer la cellule
`Ctrl+S`	enregistrer

2 Le piège de l'ordre d'exécution

```
[1]  prix = 10
```

```
[3]  total = prix * 3
     total
```

```
[2]  prix = 25
```

`total` vaut 75 et non 30 : la troisième cellule a été exécutée avant la deuxième.

- Le noyau garde en mémoire toutes les variables, quel que soit l'ordre d'exécution
- Le numéro entre crochets indique l'ordre réel
- Un notebook peut fonctionner chez soi et échouer chez un collègue

Avant de partager : Kernel, puis Restart Kernel and Run All Cells.

2 Pratique : TP1, partie 0

Mise en place de l'environnement, dans le dossier `exercices/tp01-environnement-pandas`

1	2	3	4	5	6
Créer	**Activer**	**Installer**	**Figer**	**Lancer**	**Vérifier**
`python3 -m venv .venv`	`source .venv/bin/activate`	jupyterlab, pandas, numpy, openpyxl, pyarrow	le `requirements.txt` avec `pip freeze`	`jupyter lab`, puis ouvrir `tp01.ipynb`	questions 0.1 et 0.2

`sys.executable` doit pointer vers le `.venv` du TP. Sinon, Jupyter a été lancé hors de l'environnement : `Ctrl+C`, activer, relancer.

03

Python intermédiaire

Compréhensions | lambda | f-strings | assert

3 Les compréhensions de liste

- Construire une nouvelle liste à partir d'un itérable, en une seule expression

Syntaxe : `[expression for x in iterable if condition]`

- La condition est facultative et sert à filtrer
- Plus concis qu'une boucle, et souvent plus rapide

Avec une boucle

```
prix_ht = [12.0, 45.5, 8.9, 120.0]

prix_ttc = []
for prix in prix_ht:
    prix_ttc.append(round(prix * 1.2, 2))
```

Avec une compréhension

```
prix_ttc = [round(prix * 1.2, 2) for prix in prix_ht]

gros_montants = [p for p in prix_ht if p > 40]
# [45.5, 120.0]
```

3 Filtrer ou transformer

Le `if` à la fin filtre

```
nombres = [1, 2, 3, 4, 5]

pairs = [n for n in nombres if n % 2 == 0]
# [2, 4]
```

Certains éléments sont retirés : la liste obtenue est plus courte.

Le `if ... else` au début transforme

```
parite = ["pair" if n % 2 == 0 else "impair"
          for n in nombres]
# ['impair', 'pair', 'impair', 'pair', 'impair']
```

Chaque élément est conservé et transformé : la liste garde la même longueur.

Au-delà d'une ligne lisible, ou avec plusieurs boucles imbriquées, une boucle classique reste préférable.

3 Dictionnaires et ensembles

Compréhension de dictionnaire

```
stock = {"clavier": 12, "souris": 0, "écran": 3}

# filtrer
en_stock = {p: q for p, q in stock.items() if q > 0}

# inverser clés et valeurs
codes = {"FR": "France", "BE": "Belgique"}
pays_vers_code = {v: k for k, v in codes.items()}

# associer deux listes parcourues en parallèle
ventes = dict(zip(produits, quantites))
```

Ensemble et expression génératrice

```
regions = ["Bretagne", "Occitanie", "Bretagne"]

# ensemble : dédoublonne, sans ordre
regions_uniques = {r for r in regions}
sorted(regions_uniques)
# ['Bretagne', 'Occitanie']

# générateur : pas de liste en mémoire
total = sum(p * 1.2 for p in prix_ht)
```

3 Les fonctions lambda

- Fonction anonyme, limitée à une seule expression
- Sert à passer une petite fonction en argument, là où elle ne sert qu'une fois

On ne l'assigne pas à une variable : autant écrire un `def`
Cet après-midi : `apply` et `map` avec pandas

Équivalence

```
def carre(x):
    return x ** 2

lambda x: x ** 2
```

Usage typique

```
ventes = [("clavier", 45), ("souris", 120), ("écran", 18)]

sorted(ventes, key=lambda v: v[1])
# [('écran', 18), ('clavier', 45), ('souris', 120)]

max(ventes, key=lambda v: v[1])
# ('souris', 120)
```

3 Les f-strings et le formatage des nombres

```
ca = 1234567.891
taux = 0.4567
produit = "clavier"

f"{ca:.2f}"          # '1234567.89'
f"{ca:,.2f}"         # '1,234,567.89'
f"{ca:,.0f}"         # '1,234,568'
f"{taux:.1%}"        # '45.7%'
f"{produit:<10}|"    # 'clavier   |'
f"{42:>6}"           # '    42'
```

Après les deux-points	Effet
`.2f`	2 décimales
`,`	séparateur de milliers
`.1%`	pourcentage, multiplié par 100
`<10`, `>6`	aligné à gauche, à droite, sur une largeur donnée

Séparateurs à l'anglaise : virgule pour les milliers, point pour les décimales.

3 Vérifier un résultat avec assert

```
def montant_ttc(montant_ht, taux_tva=0.2):
    return round(montant_ht * (1 + taux_tva), 2)

assert montant_ttc(100) == 120.0
assert montant_ttc(100, 0.055) == 105.5, \
    "le taux réduit est mal appliqué"
```

- Condition vraie : il ne se passe rien

Condition fausse : arrêt avec une `AssertionError`, accompagnée du message éventuel

- Le moyen le plus simple de contrôler un résultat connu à l'avance

Attention aux décimaux : `0.1 + 0.2 == 0.3` est faux. On compare des valeurs arrondies, `round(x, 2) == 0.3`, ou on tolère un écart, `abs(x - 0.3) < 1e-9`.

3 Pratique : TP1, partie 1

Python intermédiaire, dans le notebook `tp01.ipynb`

Questions	Ce qu'on pratique
1.1 à 1.5	compréhensions de liste, de dictionnaire et d'ensemble
1.6 et 1.7	trier et chercher un maximum avec `sorted`, `max` et une `lambda`
1.8	convertir des prix saisis à la main, avec une virgule ou un point
1.9 et 1.10	formater un résultat avec une f-string, vérifier une fonction avec `assert`

Compléter les cellules `# TODO`, comparer avec le **résultat attendu**, puis exécuter la cellule de **vérification** : elle affiche `OK`.

04

Les bases de numpy

Créer | Accéder | Calculer | Filtrer

4 Le tableau numpy

```
import numpy as np

notes = np.array([12, 8, 15, 9, 18])

notes.shape    # (5,)
notes.dtype    # dtype('int64')
```

Ce qui change par rapport à une liste

Tous les éléments ont le **même type**
Les calculs s'appliquent à **tous les éléments d'un coup**, sans boucle

- Chaque colonne d'un DataFrame pandas est un tableau numpy

4 Liste ou tableau : le même opérateur, deux résultats

Liste Python

```
[1, 2, 3] * 2
# [1, 2, 3, 1, 2, 3]

# la liste est répétée
```

Tableau numpy

```
np.array([1, 2, 3]) * 2
# array([2, 4, 6])

# chaque élément est multiplié
```

Une opération qui s'applique à chaque élément, sans boucle, est dite **vectorisée**.

4 Créer un tableau et accéder aux éléments

Créer sans écrire les valeurs

```
np.zeros(5)            # 5 zéros
np.ones(3)             # 3 uns
np.arange(5)           # 0, 1, 2, 3, 4
np.linspace(0, 1, 5)   # 0, 0.25, 0.5, 0.75, 1
```

Accéder par position

```
notes[0]      # 12, le premier
notes[-1]     # 18, le dernier
notes[1:3]    # array([ 8, 15])
```

Comme avec `range` et les listes, la borne de fin est **exclue**, sauf avec `linspace`.

4 Calculer et filtrer sans boucle

Calculer

```
notes + 1
# array([13,  9, 16, 10, 19])
prix = np.array([10.0, 25.0, 8.0])
quantites = np.array([3, 1, 10])
prix * quantites
# array([30., 25., 80.])

notes.sum()      # 62
notes.mean()     # 12.4
```

Filtrer

```
notes >= 10
# array([ True, False,  True, False,  True])

notes[notes >= 10]
# array([12, 15, 18])

(notes >= 10).sum()    # 3
```

C'est exactement ainsi qu'on filtrera les lignes d'un DataFrame cet après-midi.

4 Deux dimensions et nombres aléatoires

2 magasins, 3 jours

```
ventes = np.array([[3, 5, 2],
                   [4, 1, 6]])

ventes.shape         # (2, 3)
ventes[1, 2]         # 6
ventes.sum(axis=0)   # par jour : [7, 6, 8]
ventes.sum(axis=1)   # par magasin : [10, 11]
```

Un tirage reproductible

```
rng = np.random.default_rng(42)
rng.integers(1, 7, size=10)
# 10 lancers de dé
```

Même seed, mêmes valeurs à chaque exécution.

`axis=0` : un résultat par colonne. `axis=1` : un résultat par ligne. Même paramètre avec pandas.

4 Pratique : TP1, partie 2

numpy, dans le notebook `tp01.ipynb`

Questions	Ce qu'on pratique
2.1	liste ou tableau : le même opérateur, deux résultats
2.2 et 2.3	créer un tableau, accéder aux éléments
2.4 à 2.6	calculer et filtrer sans boucle
2.7 et 2.8	deux dimensions, tirage aléatoire avec une seed

05

Introduction à pandas

Series | DataFrame | Index

5 La Series : des valeurs et un index

Un tableau à une dimension dont chaque valeur porte un label : l'index.

```
import pandas as pd

temperatures = pd.Series(
    [12.5, 14.0, 9.8, 16.2, 11.1],
    index=["lundi", "mardi", "mercredi",
           "jeudi", "vendredi"],
    name="temperature",
)

temperatures.loc["mardi"]   # par label
temperatures.iloc[1]        # par position
```

index	temperature
lundi	12.5
mardi	14.0
mercredi	9.8
jeudi	16.2
vendredi	11.1

```
temperatures.mean()     # 12.72
temperatures.idxmax()   # 'jeudi'
```

5 Les calculs s'alignent sur l'index

L'addition associe les mêmes labels

```
ventes_lundi = pd.Series({"clavier": 3, "souris": 5})
ventes_mardi = pd.Series({"souris": 2, "écran": 1})

ventes_lundi + ventes_mardi
# clavier    NaN
# souris     7.0
# écran      NaN
```

Un label absent compte pour 0

```
ventes_lundi.add(ventes_mardi, fill_value=0)
# clavier    3.0
# souris     7.0
# écran      1.0
```

Les méthodes `sub`, `mul` et `div` fonctionnent de la même façon.

pandas calcule sur les valeurs qui portent **le même label**, pas sur celles qui sont à la même position. `NaN` (Not a Number) signale une valeur manquante.

5 Le DataFrame : un tableau de Series

index	produit	prix	quantite
0	clavier	45.0	3
1	souris	19.9	10
2	écran	189.0	1

L'index identifie les lignes, les noms de colonnes identifient les Series.

- Un tableau à deux dimensions : lignes et colonnes

Chaque colonne est une **Series**, avec son propre type
Toutes les colonnes partagent le **même index**

- Par défaut, l'index est numérique et commence à 0

5 Créer et parcourir un DataFrame

À partir d'un dictionnaire de listes

```
df = pd.DataFrame({
    "produit": ["clavier", "souris",
"écran"],
    "prix": [45.0, 19.9, 189.0],
    "quantite": [3, 10, 1],
})
```

Ou d'une liste de dictionnaires

```
pd.DataFrame([
    {"produit": "clavier", "quantite": 3},
    {"produit": "souris"},   # quantite : NaN
])
```

Premiers accès

```
df["prix"]                  # Series
df[["produit", "prix"]]     # DataFrame

df.columns
df.index

df["montant"] = df["prix"] * df["quantite"]
```

`df["prix"]` renvoie une Series, `df[["prix"]]` renvoie un DataFrame d'une seule colonne.

5 Un DataFrame à partir d'un tableau numpy

```
rng = np.random.default_rng(1)
visites = pd.DataFrame(
    rng.integers(0, 100, size=(4, 3)),
    columns=["lundi", "mardi", "mercredi"],
    index=["A", "B", "C", "D"],
)

visites.sum()           # un total par colonne
visites.sum(axis=1)     # un total par ligne
```

	lundi	mardi	mercredi
A	47	51	75
B	95	3	14
C	82	94	24
D	31	86	42

`columns` nomme les colonnes, `index` les lignes. `axis` fonctionne comme avec numpy.

5 Listes, dictionnaires, Series et DataFrame

	Liste	Dictionnaire	Series	DataFrame
Dimensions	1	1	1	2
Accès	par position	par clé	par label et par position	par colonne, par ligne, ou les deux
Types	mélangés	mélangés	un type par Series	un type par colonne
Calcul vectorisé	non	non	oui	oui, par colonne
Valeurs manquantes	non prévu	non prévu	`NaN` géré	`NaN` géré

Une Series ressemble à un dictionnaire ordonné sur lequel on calcule comme avec numpy. Un DataFrame ressemble à un dictionnaire de Series qui partagent le même index.

5 Pratique : TP1, partie 3

Series et DataFrame, dans le notebook `tp01.ipynb`

Questions	Ce qu'on pratique
3.1 à 3.3	créer une Series, `loc` et `iloc`, moyenne, `idxmax`, filtre booléen
3.4	additionner deux Series, puis `add` avec `fill_value`
3.5 à 3.7	créer un DataFrame, calculer des colonnes, Series ou DataFrame ?
3.8 et 3.9	un DataFrame depuis une liste de dictionnaires, puis depuis un tableau numpy

06

Lire et écrire des fichiers

CSV | Excel | Parquet

6 Lire un CSV avec read_csv

```
df = pd.read_csv("data/ventes.csv")
```

- Le chemin est relatif au dossier de lancement de Jupyter

`FileNotFoundError`: vérifier d'abord ce dossier

- Une option pour presque chaque particularité du fichier

Option	Exemple
`sep`	`sep=";"`
`decimal`	`decimal=","`
`encoding`	`encoding="cp1252"`
`usecols`	`usecols=["date", "prix"]`
`dtype`	`dtype={"code_postal": str}`
`parse_dates`	`parse_dates=["date"]`
`index_col`	`index_col="id_commande"`
`na_values`	`na_values=["", "N/A", "-"]`
`nrows`	`nrows=1000`
`skiprows`	`skiprows=2`

6 Le cas du CSV exporté depuis Excel

```
df = pd.read_csv(
    "data/export_excel.csv",
    sep=";",
    decimal=",",
    encoding="cp1252",
)
```

Symptôme	Option à ajuster
Une seule colonne, avec tous les en-têtes collés	`sep`
Des nombres lus comme du texte : `"420,5"`	`decimal`
Une `UnicodeDecodeError`, ou des accents illisibles comme `Ã©`	`encoding`
Des zéros perdus : `01000` devient `1000`	`dtype`
Des dates lues comme du texte	`parse_dates`

6 Écrire un CSV, lire et écrire de l'Excel

CSV

```
df.to_csv("data/ventes_export.csv", index=False)
```

Sans `index=False`, l'index est écrit comme première colonne et revient à la relecture sous le nom `Unnamed: 0`.

Excel (nécessite openpyxl)

```
pd.read_excel("data/ventes.xlsx", sheet_name="2025")
pd.read_excel("data/ventes.xlsx", sheet_name=None)

df.to_excel("data/ventes.xlsx", index=False)

with pd.ExcelWriter("data/rapport.xlsx") as w:
    df_2024.to_excel(w, sheet_name="2024")
    df_2025.to_excel(w, sheet_name="2025")
```

`sheet_name=None` renvoie un dictionnaire : un DataFrame par feuille.

6 Parquet : le format de travail

Nécessite pyarrow

```
df.to_parquet("data/ventes.parquet",
              index=False)

pd.read_parquet("data/ventes.parquet")

pd.read_parquet(
    "data/ventes.parquet",
    columns=["date", "region"],
)
```

Lecture rapide, colonne par colonne. En contrepartie, il ne s'ouvre ni dans un éditeur de texte ni dans Excel.

1

Les types sont conservés

Une date reste une date, sans rien préciser à la relecture.

2

Des fichiers compressés

Souvent plusieurs fois plus légers que le CSV équivalent.

6 Quel format choisir ?

	CSV	Excel	Parquet
Lisible par un humain	oui, éditeur de texte	oui, Excel	non
Types conservés	non	en partie	oui
Taille	grande	moyenne	petite
Vitesse de lecture	moyenne	lente	rapide
Usage type	échange universel	échange avec les métiers	stockage et traitement internes

6 Pratique : TP1, partie 4

Lecture et écriture de fichiers, dans le notebook `tp01.ipynb`

Questions	Ce qu'on pratique
4.1 à 4.3	lire les ventes, convertir les dates, ne lire que certaines colonnes et lignes
4.4 et 4.5	lire un export de tableur français : séparateur, décimale, codes postaux
4.6 et 4.7	écrire en CSV, Excel et Parquet, puis comparer les types à la relecture

Question 4.4 : lire d'abord le fichier sans option, observer le symptôme, puis choisir les options.

07

Première exploration

shape | info | describe | value_counts | nunique

7 Une commande pour chaque question

Commande	Question
`df.shape`	combien de lignes et de colonnes ?
`df.head(10)`, `df.tail()`	à quoi ressemblent les premières et les dernières lignes ?
`df.sample(5, random_state=0)`	et quelques lignes prises au hasard ?
`df.dtypes`, `df.info()`	quel type pour chaque colonne, combien de valeurs non nulles ?
`df.describe()`	quelles statistiques pour les colonnes numériques ?
`df["col"].value_counts()`	comment se répartissent les valeurs d'une colonne ?
`df["col"].nunique()`, `df.nunique()`	combien de valeurs distinctes ?
`df.isna().sum()`	combien de valeurs manquantes par colonne ?

7 Lire le résultat de info()

```
RangeIndex: 62 entries, 0 to 61
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id_commande     62 non-null     int64
 1   date            62 non-null     str
 2   region          59 non-null     str
 3   categorie       62 non-null     str
 4   produit         62 non-null     str
 5   Prix unitaire   62 non-null     float64
 6   quantite        62 non-null     int64
 7   Client fidèle   62 non-null     bool
```

À lire en priorité

Les valeurs non nulles : 59 sur 62 pour `region`, donc trois valeurs manquantes
Les types : `date` en `str` est du texte, pas une date. Il faudra la convertir
Les noms de colonnes : `Prix unitaire` et `Client fidèle` ne suivent pas la convention des autres

7 describe et les valeurs aberrantes

```
df.describe()

# inclut aussi les colonnes texte
df.describe(include="all")
```

- Pour chaque colonne numérique :
  - nombre de valeurs, moyenne, écart-type
  - minimum, quartiles, maximum

Le moyen le plus rapide de repérer une valeur **impossible** : un prix négatif, un âge de 250 ans
Comparer la moyenne et la médiane (ligne `50%`) : un écart important signale quelques valeurs extrêmes

7 value_counts pour les colonnes catégorielles

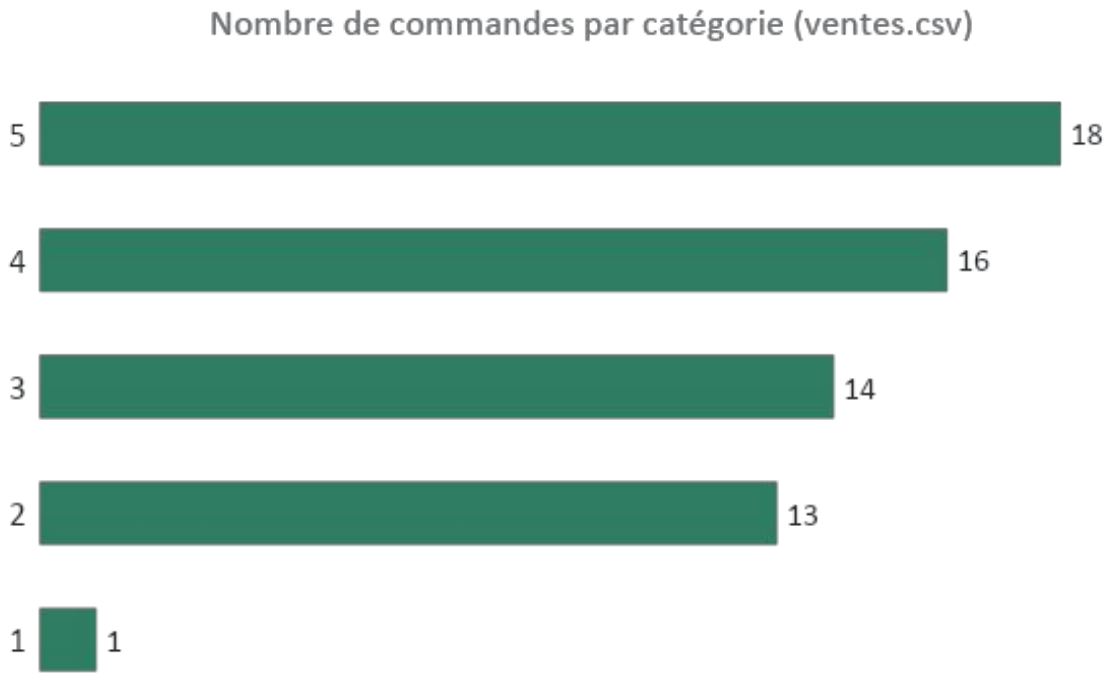
```
df["categorie"].value_counts()

# en proportions
df["categorie"].value_counts(normalize=True)

# en comptant les manquants
df["region"].value_counts(dropna=False)

# combinaisons de plusieurs colonnes
df[["categorie", "produit"]].value_counts()
```

Permet aussi de repérer les incohérences d'écriture : `Bretagne`, `bretagne` et `Bretagne ` comptent comme trois valeurs.



7 nunique : combien de valeurs distinctes ?

```
df.nunique()

df["id_commande"].nunique()    # 60
len(df)                        # 62

# les identifiants en double
df["id_commande"].value_counts().head(3)
```

```
id_commande       60
date              53
region             9
categorie          5
produit           16
Prix unitaire     60
quantite          14
Client fidèle      2
dtype: int64
```

62 lignes, mais 60 numéros de commande : certaines commandes apparaissent plusieurs fois.

7 Une démarche d'exploration

1	2	3	4	5	6	7
Charger	**Mesurer**	**Typer**	**Compter les manques**	**Décrire**	**Répartir**	**Noter**
lecture correcte ?	`shape`, `head`	`dtypes`, `info`	`isna().sum()`	`describe`	`value_counts`, `nunique`	constats et questions

Le résultat de cette étape n'est pas un tableau propre, c'est une liste de constats et de questions. C'est elle qui guidera le nettoyage de cet après-midi.

7 Pratique : TP1, partie 5 et synthèse

Exploration du jeu de données des ventes, dans le notebook `tp01.ipynb`

Questions	Ce qu'on pratique
5.1 à 5.3	dimensions, `info`, `isna`, `describe` : manques et valeurs suspectes
5.4 à 5.6	`value_counts` et `nunique` : variantes d'écriture, commandes en double
5.7 et 5.8	questions rapides, puis recherche libre d'autres anomalies
Synthèse	dimensions, types à corriger, manques, aberrantes, incohérences, questions

La synthèse, rédigée dans le notebook, sert de point de départ au nettoyage de cet après-midi.

8 TP1 : environnement et première exploration

Le notebook `exercices/tp01-environnement-pandas/tp01.ipynb`, complété au fil de la matinée

1

Mettre en place

Partie 0 : un environnement virtuel avec venv, les bibliothèques avec pip, Jupyter lancé depuis l'environnement.

2

Pratiquer

Parties 1 à 4 : Python intermédiaire, numpy, Series et DataFrame, lecture et écriture de fichiers.

3

Explorer

Partie 5 : explorer les ventes, puis rédiger la synthèse : dimensions, types, valeurs aberrantes, questions à creuser.

Livrable : un environnement virtuel avec son `requirements.txt`, et le notebook d'exploration commenté, qui s'exécute de haut en bas.

À retenir

Ubuntu : `python3` hors environnement, `python` dans le `.venv` ; `pip install` toujours dans un environnement virtuel
Un projet, un `.venv`, un `requirements.txt`. Le `.venv` ne se partage pas

- Avant de partager un notebook : tout réexécuter
- Compréhensions : concises, tant qu'elles restent lisibles

`lambda` : une petite fonction passée en argument
f-strings pour formater (`:,`, `.2f`, `:.1%`), `assert` pour vérifier

- numpy : un tableau d'un seul type, on calcule et on filtre sans boucle

`axis=0` : un résultat par colonne. `axis=1` : par ligne

- Series : une colonne avec un index. DataFrame : des Series qui partagent l'index

`read_csv` : `sep`, `decimal`, `encoding`, `dtype`, `parse_dates`

- Parquet conserve les types et pèse moins lourd

Toujours commencer par `shape`, `info`, `describe`, `isna`, `value_counts`, `nunique`